

AI-Based Household Wealth Estimation System Using Docker Compose and CI/CD

DEVOPS PROJECT REPORT

Submitted By
Kunal Mahore

B.Tech Computer Science Engineering
Academic Year: 2025–26

Table of Contents

Chapter 1 — Introduction

1.1 Project Overview

1.2 Problem Statement

1.3 Objectives

1.4 Scope

Chapter 2 — System Requirements

2.1 Hardware Requirements

2.2 Software Requirements

Chapter 3 — Workflow

Chapter 4 — Implementation

4.1 Setup Steps

4.2 Configurations

4.3 Commands Used

4.4 Dockerfile

4.5 Docker Compose

4.6 GitHub Actions CI/CD

4.7 GitHub Repository

Chapter 5 — Results and Output

Chapter 6 — Future Scope

Chapter 7 — Conclusion

Chapter 1 — Introduction

1.1 Project Overview

The AI-Based Household Wealth Estimation System is a DevOps-oriented deployment project that combines Artificial Intelligence, Computer Vision, Docker containerization, Docker Compose orchestration, and CI/CD automation.

The system predicts household wealth categories by analyzing exterior images of houses. The project uses an EfficientNet-B0 deep learning model trained using PyTorch. The prediction categories include low wealth, medium wealth, and high wealth.

The project was designed using a microservices architecture where the frontend and backend run as separate containers. The frontend was developed using Streamlit and the backend was developed using FastAPI. Docker Compose manages both services and provides automatic networking and orchestration.

GitHub Actions was used to automate Continuous Integration and Continuous Deployment workflows. The CI/CD pipeline automatically builds Docker images and pushes them to Docker Hub whenever code changes are pushed to the repository.

The major objective of this project is to demonstrate how modern DevOps practices can simplify deployment, improve reproducibility, and automate delivery pipelines for AI applications.

1.2 Problem Statement

Traditional Machine Learning projects are usually executed in notebooks or local environments. These projects often face problems such as dependency conflicts, inconsistent environments, deployment difficulties, and lack of scalability.

Deploying AI applications manually requires installing dependencies individually on each system. This process becomes difficult to manage and error-prone.

The project solves these problems using DevOps technologies such as Docker, Docker Compose, GitHub Actions, and CI/CD automation. The application can now run consistently across multiple systems without dependency issues.

1.3 Objectives

The objectives of the project are:

- To implement Docker containerization for AI services.
- To develop a microservices-based architecture.
- To create isolated frontend and backend services.
- To implement Docker Compose orchestration.
- To automate image builds using GitHub Actions.
- To implement CI/CD workflows.
- To improve deployment reproducibility.
- To demonstrate modern DevOps practices.
- To integrate Artificial Intelligence with deployment automation.

1.4 Scope

The project scope includes:

- AI image classification
- Containerized deployment
- Docker networking
- Docker Compose orchestration
- CI/CD automation
- Docker Hub deployment
- Frontend-backend communication
- Automated build workflows

The project can be further extended using Kubernetes, cloud deployment platforms, monitoring systems, and scalable infrastructure.

Chapter 2 — System Requirements

2.1 Hardware Requirements

Component	Requirement
Processor	Intel Core i5 or equivalent
RAM	8 GB minimum
Storage	20 GB free space
Internet	Required
GPU	Optional

2.2 Software Requirements

Software	Purpose
Python 3.11	Programming language
Docker	Containerization
Docker Compose	Container orchestration
Git	Version control
GitHub	Repository hosting
FastAPI	Backend framework
Streamlit	Frontend framework
PyTorch	Deep learning framework
GitHub Actions	CI/CD automation

Chapter 3 — Workflow

Step 1 — Dataset Collection

House exterior image datasets were collected and categorized into low, medium, and high wealth classes.

Step 2 — Data Preprocessing

Images were resized, normalized, and converted into tensors using PyTorch transformations.

Step 3 — Model Training

The EfficientNet-B0 convolutional neural network model was trained using PyTorch. The model learned image patterns related to wealth estimation.

Step 4 — Model Saving

The trained model was saved as a .pth file for deployment.

Step 5 — Backend Development

A FastAPI backend service was developed to load the model, receive image uploads, preprocess images, and return predictions.

Step 6 — Frontend Development

A Streamlit frontend was developed for image upload, prediction visualization, and confidence score display.

Step 7 — Docker Containerization

Separate Dockerfiles were created for frontend and backend services.

Step 8 — Docker Compose Orchestration

Docker Compose orchestrates frontend and backend containers and creates shared networking automatically.

Step 9 — CI/CD Automation

GitHub Actions automatically builds Docker images and pushes them to Docker Hub whenever code changes are pushed to GitHub.

Step 10 — Deployment

The project runs using docker compose up command which launches all services automatically.

Chapter 4 — Implementation

4.1 Setup Steps

1. Install Docker and Docker Compose.
2. Clone the GitHub repository.
3. Create frontend and backend requirements files.
4. Build Docker images.
5. Run containers using Docker Compose.
6. Open frontend in browser.

4.2 Configurations

Frontend Port: 8501

Backend Port: 8000

Docker Compose automatically creates a shared bridge network. The frontend communicates with backend using service name based DNS resolution.

Example:

`http://backend:8000`

4.3 Commands Used

`docker compose build`

`docker compose up`

`docker compose up --build`

`docker compose down`

`docker ps`

`docker images`

`docker network ls`

`docker compose logs`

4.4 Dockerfile

Separate Dockerfiles were created for frontend and backend services.

Backend Dockerfile responsibilities:

- Install backend dependencies
- Copy source code
- Expose backend port
- Start FastAPI service

Frontend Dockerfile responsibilities:

- Install Streamlit dependencies
- Copy frontend source code
- Expose frontend port
- Start Streamlit application

4.5 Docker Compose

Docker Compose manages multiple containers using a single YAML configuration file.

Responsibilities:

- Build frontend and backend images
- Create containers
- Create shared bridge network
- Manage startup dependencies
- Expose ports
- Enable inter-container communication

4.6 GitHub Actions CI/CD Workflow

GitHub Actions automates Continuous Integration and Continuous Deployment workflows.

Workflow steps:

- Checkout repository
- Login to Docker Hub
- Build backend image
- Build frontend image
- Push images to Docker Hub

The pipeline runs automatically whenever code is pushed to the main branch.

4.7 GitHub Repository

The complete project source code was managed using Git and GitHub.

The repository contains:

- Frontend source code
- Backend source code
- Dockerfiles
- Docker Compose configuration
- GitHub Actions workflow
- Documentation
- README file

Backend Dockerfile

```
wealth-estimation-devops > backend > Dockerfile > ...
1 FROM python:3.11-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6
7 RUN pip install --no-cache-dir -r requirements.txt
8
9 COPY . .
10
11 EXPOSE 8000
12
13 CMD ["uvicorn", "backend.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Frontend Dockerfile

```
wealth-estimation-devops > frontend > Dockerfile > ...
1 FROM python:3.11-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6
7 RUN pip install --no-cache-dir -r requirements.txt
8
9 COPY . .
10
11 EXPOSE 8501
12
13 CMD ["streamlit", "run", "frontend/app.py", "--server.address=0.0.0.0"]
```

Docker Compose File

```
wealth-estimation-devops > 🔥 docker-compose.yml
  ▶ Run All Services
1  services:
2
3    ▶ Run Service
4    backend:
5      build:
6        context: .
7        dockerfile: backend/Dockerfile
8
9      container_name: wealth-backend
10
11     ports:
12       - "8000:8000"
13
14    ▶ Run Service
15    frontend:
16      build:
17        context: .
18        dockerfile: frontend/Dockerfile
19
20      container_name: wealth-frontend
21
22     ports:
23       - "8501:8501"
24
25     depends_on:
26       - backend
```

CI Config File

```
wealth-estimation-devops > .github > workflows > ci.yml
1  name: AI DevOps CI Pipeline
2
3  on:
4    push:
5      branches:
6        - main
7
8    pull_request:
9      branches:
10       - main
11
12  jobs:
13
14    build-and-test:
15
16      runs-on: ubuntu-latest
17
18      steps:
19
20        - name: Checkout Repository
21          uses: actions/checkout@v4
22
23        - name: Set Up Python
24          uses: actions/setup-python@v5
25          with:
26            python-version: "3.11"
27
28        - name: Install Dependencies
29          run: |
30            python -m pip install --upgrade pip
31            pip install -r requirements.txt
32
33        - name: Validate Docker Compose
34          run: docker compose config
35
36        - name: Build Backend Container
37          run: |
38            docker build -t wealth-backend \
39              -f backend/Dockerfile .
40
41        - name: Build Frontend Container
42          run: |
43            docker build -t wealth-frontend \
44              -f frontend/Dockerfile .
45
46        - name: CI Pipeline Success
47          run: echo "CI pipeline completed successfully!"
```

Chapter 5 — Results and Output

The project was successfully implemented and tested.

Results achieved:

- Docker images were successfully created.
- Frontend and backend containers were launched successfully.
- Docker Compose orchestration worked correctly.
- Frontend communicated with backend through Docker networking.
- GitHub Actions CI/CD workflow executed successfully.
- Docker images were pushed to Docker Hub.
- The AI model generated wealth predictions successfully.

Example Prediction Output:

Prediction: High

Confidence: 97%

Screenshots:

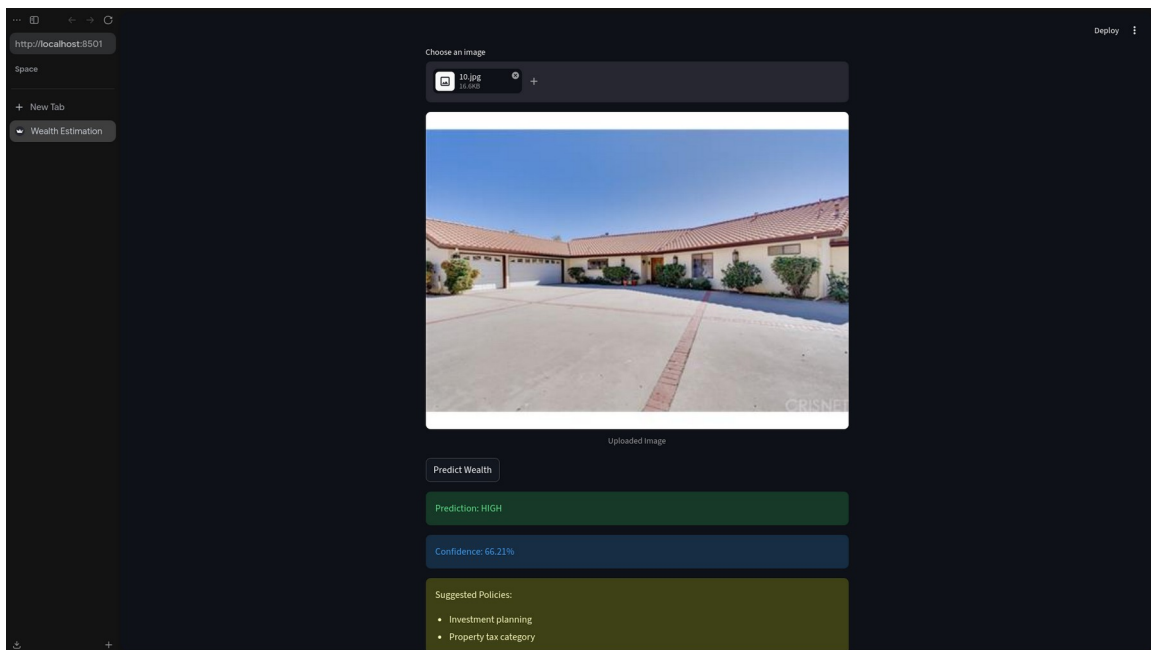
- Docker image build output

```
=> => exporting attestation manifest sha256:f6ea2150af0cc4abf1812a2d541c1b53ba241d0aea52498f02da7
=> => exporting manifest list sha256:d8d1e5ae00c8db8c470d6ba28b7170e6a2ab0a53bfa63b196d1e1876a847
=> => naming to docker.io/library/wealth-estimation-devops-frontend:latest
=> => unpacking to docker.io/library/wealth-estimation-devops-frontend:latest
=> [backend] exporting to image
=> => exporting layers
=> => exporting manifest sha256:72fda036a16ee2904babc1fe250e9d5715e7cc824cd2da3958e5031f05f276cc
=> => exporting config sha256:030acd0d53a5d19c77dad447b460e4f555988beb24e4541e95b08d4667cb20c2
=> => exporting attestation manifest sha256:62ee8768d1e5b7a1ae75a1cdcbc42bb0ca766cf7eb8fe66dd8e87
=> => exporting manifest list sha256:e8d8cc56ed31d17a99172a7f50d63c54519a4f5c565428a0b3dbf63997ba
=> => naming to docker.io/library/wealth-estimation-devops-backend:latest
=> => unpacking to docker.io/library/wealth-estimation-devops-backend:latest
=> [frontend] resolving provenance for metadata file
=> [backend] resolving provenance for metadata file
+] build 2/2
✓ Image wealth-estimation-devops-backend Built
✓ Image wealth-estimation-devops-frontend Built
```

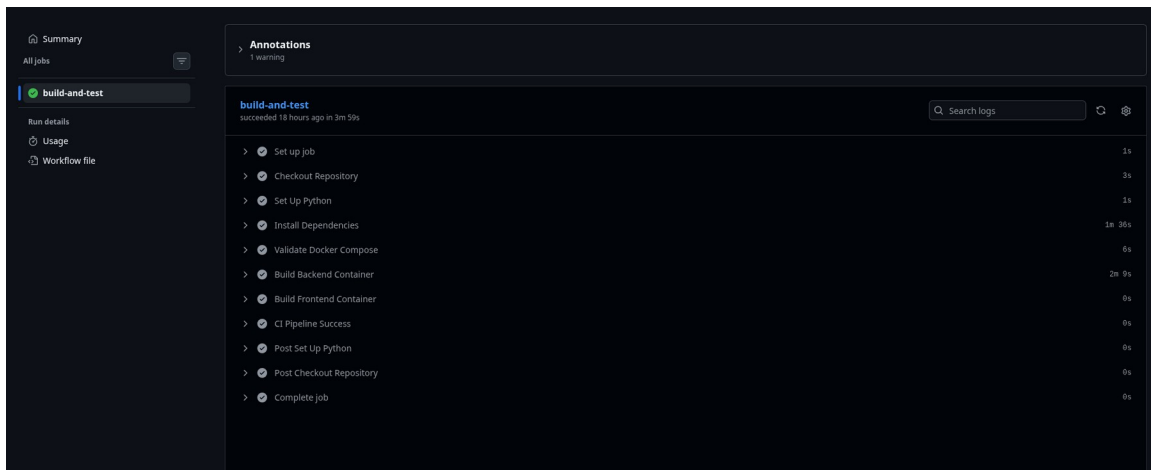
- Running containers

```
wealth-estimation-devops docker compose up
Attaching to wealth-backend, wealth-frontend
wealth-frontend |
wealth-frontend | Collecting usage statistics. To deactivate, set browser.gatherUsageStats to false.
wealth-frontend |
wealth-frontend | 2026-05-30 11:28:10.273 Uvicorn server started on 0.0.0.0:8501
wealth-frontend |
wealth-frontend | You can now view your Streamlit app in your browser.
wealth-frontend |
wealth-frontend | Local URL: http://localhost:8501
wealth-frontend | Network URL: http://172.18.0.3:8501
wealth-frontend | External URL: http://128.185.168.207:8501
wealth-frontend |
```

- Streamlit frontend



- GitHub Actions workflow success



- Docker Hub repositories

Chapter 6 — Future Scope

Future improvements include:

- Kubernetes orchestration
- Cloud deployment using AWS or Azure
- Jenkins integration
- Monitoring dashboards
- Auto-scaling infrastructure
- GPU acceleration
- Database integration
- User authentication
- Production-grade deployment

The project can be transformed into a large-scale AI deployment platform using modern DevOps technologies.

Chapter 7 — Conclusion

The project successfully demonstrated how Artificial Intelligence applications can be integrated with modern DevOps practices.

The project achieved:

- Docker containerization
- Docker Compose orchestration
- CI/CD automation
- Docker networking
- Automated deployment workflows
- Microservices architecture

Technologies learned:

- Docker
- Docker Compose
- GitHub Actions
- FastAPI
- Streamlit
- PyTorch
- Container orchestration
- CI/CD automation

The final outcome was a fully containerized AI deployment system capable of automated builds and scalable execution using DevOps methodologies.